

FUTURE IN DART

Future In Dart

In dart, the Future represents a value or error that is not yet available. It is used to represent a potential value, or error, that will be available at some time in the future.

How To Create Future In Dart

You can create a future in dart by using **Future** class. Here the function will return

`Future<String>`  after 5 seconds.

```
// function that returns a future
Future<String> getUsername() async {
  return Future.delayed(Duration(seconds: 2), () => 'Mark');
}
```



You can also create a future by using `Future.value()`  method. Here the function will return `Future<String>`  immediately.

```
// function that returns a future
Future<String> getUsername() {
  return Future.value('Mark');
}
```



How To Use Future In Dart

You can use future in dart by using `then()`  method. Here the function will return

`Future<String>`  after 5 seconds.

```
// function that returns a future
Future<String> getUsername() async {
  return Future.delayed(Duration(seconds: 2), () => 'Mark');
}

// main function
```



```
void main() {  
  print("Start");  
  getUsername().then((value) => print(value));  
  print("End");  
}
```

[➤ Show Output](#)

More About Future

Future represents the result of an asynchronous operation and can have 2 states.

State Of Future

- **Uncompleted**
- **Completed**

Uncompleted

When you call an asynchronous function, it returns to an uncompleted future. It means the future is waiting for the function asynchronous operation to finish or to throw an error.

Completed

It can be completed with value or completed with error. `Future<int>` produces an int value, and `Future<String>` produces a String value. If the future doesn't produce any value, then the type of future is `Future<void>`.

Info

Note: If the asynchronous operation performed by the function fails due to any reason, the future completes with an error.

Example 2: Future In Dart

In this example below, we are creating a function **middleFunction()** that returns a future. The function will return `Future<String>` after 5 seconds.



```
void main() {  
    print("Start");  
    getData();  
    print("End");  
}  
  
void getData() async{  
    String data = await middleFunction();  
    print(data);  
}  
  
Future<String> middleFunction(){  
    return Future.delayed(Duration(seconds:5), ()=> "Hello");  
}
```

> Show Output

Info

Note: In the above example, First, it prints **Start**, secondly it prints **End**, and after 5 seconds **Hello** will be printed.